

Yenepoya Institute Of Arts, Science, Commerce & Management

Project – High Level Design

on

Natural Language Interface for Supply Chain Infrastructure

Student Name: Gokul Vinod M

RegNo: 23BCAICD042

Industry Mentor: Mr Sumit Kumar

Guided by: Ms Manjula

Table of Contents

1. Introduction.....	3
1.1. Scope of the document.....	3
1.2. Intended Audience.....	3
1.3. System overview.....	3
2. System Design.....	4
2.1. Application Design.....	4
2.2. Process Flow.....	4
2.3. Information Flow.....	5
2.4. Components Design.....	5
2.5. Key Design Considerations.....	6
2.6. API Catalogue.....	7
3. Data Design.....	7
3.1. Data Model.....	7
3.2. Data Access Mechanism.....	8
3.3. Data Retention Policies.....	8
3.4. Data Migration.....	8
4. Interfaces.....	8
5. State and Session Management.....	9
6. Caching.....	9
7. Non-Functional Requirements.....	9
7.1. Security Aspects.....	9
7.2. Performance Aspects.....	9
8. References.....	10

1. INTRODUCTION

The Supply Chain Management Chatbot is a Python-based command-line application that helps users access supply chain data like orders, shipments, inventory, and suppliers. It uses keyword-based input processing to provide quick and relevant responses from JSON data.

1.1 Scope of the Document

This document provides a high-level architectural overview of the **Supply Chain Management Chatbot**. It outlines system components, data flow, design considerations, and overall structure.

1.2 Intended Audience

- Developers
- Project Evaluators
- System Designers
- Academic Reviewers

1.3 System Overview

The Supply Chain Management Chatbot is a command-line-based application that enables users to retrieve supply chain-related information such as:

- Orders
- Shipments
- Inventory
- Suppliers
- Warehouses
- Customers

- Products
- Transport

The chatbot uses a rule-based Natural Language Processing (NLP) approach to interpret user inputs and provide responses.

2. SYSTEM DESIGN

2.1 Application Design

The system follows a modular architecture consisting of:

- Main Interface (CLI Layer)
Handles user interaction via terminal
- Parser Module
Processes user input and determines intent
- Response Module
Fetches and formats data
- Data Layer
JSON-based storage for all supply chain data

2.2 Process Flow

- User enters a command
- Input is converted to lowercase
- Parser analyzes keywords
- Appropriate function is triggered
- Data is fetched from JSON files
- Response is displayed to the user

2.3 Information Flow

User Input → Parser → Response Functions → JSON Data → Output to User

2.4 Components Design

2.4.1. Main Module

- Runs chatbot loop
- Handles exit condition
- Calls parser

2.4.2. Parser Module (process input)

- Identifies keywords like:
order, shipment, inventory, supplier, etc.
- Routes to corresponding functions

2.4.3. Response Module

Contains functions such as:

- order_status()
- shipment_status()
- inventory_status()
- supplier_info()
- warehouse_info()
- customer_info()
- product_info()
- transport_info()

2.4.4. Data Loader

- Loads JSON data using `json.load()`
- Files include:
 - `orders.json`
 - `shipments.json`
 - `inventory.json`
 - `suppliers.json`
 - `warehouses.json`
 - `deliveries.json`
 - `customers.json`
 - `products.json`
 - `transport.json`

2.5 Key Design Considerations

- Simplicity: Easy-to-use CLI interface
- Modularity: Separate parser and response logic
- Scalability: Easy to add new commands
- Maintainability: JSON-based data storage
- Performance: Fast lookup using in-memory data

2.6 API Catalogue

Function Name	Description
process_input()	Parses user input
help_menu()	Displays commands
order_status()	Shows order details
shipment_status()	Shows shipment tracking
inventory_status()	Displays stock levels
supplier_info()	Lists suppliers
warehouse_info()	Shows warehouse details
delivery_schedule()	Displays delivery plan
customer_info()	Lists customers
product_info()	Shows products
transport_info()	Displays transport details

3. DATA DESIGN

The system stores data in JSON files such as orders, inventory, and shipments. The data is loaded into memory and accessed by functions to generate responses. This approach is simple and easy to manage.

3.1 Data Model

Data is stored in JSON format:

- Orders → ID, Status
- Shipments → Tracking, Location
- Inventory → Product, Quantity
- Suppliers → Name, Product
- Warehouses → ID, Location, Capacity

- Customers → ID, Name, City
- Products → ID, Name, Price
- Transport → Vehicle ID, Type, Capacity, Driver

3.2 Data Access Mechanism

- JSON files loaded at runtime
- Stored in memory as Python objects
- Accessed via iteration

3.3 Data Retention Policies

- Static dataset (no real-time updates)
- Data persists in JSON files
- No deletion or modification logic

3.4 Data Migration

- Can be upgraded to:
 - i. SQL Database
 - ii. Cloud Storage
- JSON → Database migration supported easily

4. INTERFACES

- User Interface
Command-line interface (CLI)
- System Interface
Python modules interaction

- Data Interface
JSON file reading

5. STATE AND SESSION MANAGEMENT

- Stateless system
- Each input is processed independently
- No session storage

6. CACHING

- No caching implemented
- Entire dataset loaded at runtime
- Fast access due to small dataset

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Security Aspects

- No authentication required
- Safe local execution
- No external API calls

7.2 Performance Aspects

- Fast response time
- Low memory usage
- Efficient keyword matching

8. REFERENCES

- Python Documentation
- JSON Data Handling
- Google
- Rule-Based Chatbot Design Concepts